UET NOWSHERA | ELECTRICAL ENGINEERING

LAB REPORT 13

STRUCTURES IN C++

COURSE: Computer Programming

AUTHOR: Adnan Ahmad.

ROLL NO: BF25NWELE0744

DATE: 14/5/26

1. OBJECTIVE

- To understand how to group different data types into a single unit using structures.
- To learn how to declare and define a struct in C++.
- To create structure variables and access their individual members.
- To initialize structures and pass structured data effectively.

2. THEORETICAL BACKGROUND

What is a Structure? A Structure is a collection of simple variables. The variables in a structure can be of different types: some can be int, some can be float, and so on. This is unlike an array, in which all the variables must be of the same type. The data items within a structure are referred to as the members of the structure.

Structure Declaration. A structure is declared using the keyword 'struct' followed by a

structure name (or tag). Inside the curly braces, the members are defined. It is crucial to note that the closing brace in the structure type declaration must be followed by a semicolon. A structure declaration simply defines the 'form' or blueprint; it does not reserve memory until a variable of that structure type is instantiated.

Accessing Members. To access or assign values to the individual members of a structure variable, the dot (.) operator is used. For example, if 'part1' is a structure variable and 'cost' is a member, it is accessed as 'part1.cost'.

Nested Structures & Copying. One structure can be nested within another. Furthermore, entire structure variables can be assigned to one another (e.g., part2 = part1;), which copies all member values from the source structure to the destination structure in a single operation.

3. SOFTWARE USED

- IDE: DEV-C++ IDE
- Compiler: C++ Compiler (bundled with DEV-C++)

4. EXPERIMENTAL TASKS

Task 1: Structure Declaration (Person)

A structure named Person was declared containing a name, age, and address. A variable of this structure type was initialized, and its individual members were populated and displayed using dot notation.

```
#include "str"><iostream>
#include "str"><string>
using namespace std;

// Declare the structure
struct Person {
    string name;
    int age;
    string address;
};

int main() {
    // Declare and initialize a structure variable
    Person person1;
    person1.name = "Qazi Ehsanullah";
    person1.age = 22;
    person1.address = "Peshawar, Pakistan";

    // Display the members
    cout << "--- Person Details ---" << endl;
    cout << "Name: " << person1.name << endl;
    cout << "Age: " << person1.age << endl;
    cout << "Address: " << person1.address << endl;

    return 0;
}
```

Task 2: Structure Declaration (Student)

A structure named Student was created with members for name, age, and grade. A variable named student1 was declared, its data was assigned, and the contents were printed to the screen.

```
#include "str"><iostream>
#include "str"><string>
using namespace std;

// Declare the structure
struct Student {
    string name;
    int age;
    char grade;
};

int main() {
    // Declare and initialize using an initializer list
    Student student1 = {"Ali Khan", 20, 'A'};

    // Display the members
    cout << "--- Student Details ---" << endl;
    cout << "Name: " << student1.name << endl;
    cout << "Age: " << student1.age << endl;
    cout << "Grade: " << student1.grade << endl;

    return 0;
}
```

Task 3: Accessing Multiple Structure Elements

A structure named Book was declared containing title, author, price, and pages. Two distinct variables of this structure type (book1 and book2) were instantiated, assigned different values, and subsequently displayed side-by-side to demonstrate managing multiple records.

```
#include "str"><iostream>
#include "str"><string>
using namespace std;

// Declare the structure
struct Book {
    string title;
    string author;
    float price;
    int pages;
};

int main() {
    // Declare and initialize two structure variables
    Book book1 = {"C++ Primer", "Stanley Lippman", 45.50, 976};
    Book book2;

    book2.title = "Clean Code";
    book2.author = "Robert C. Martin";
    book2.price = 38.99;
    book2.pages = 464;

    // Display book1
    cout << "--- Book 1 ---" << endl;
    cout << "Title: " << book1.title << endl;
    cout << "Author: " << book1.author << endl;
    cout << "Price: $" << book1.price << endl;
    cout << "Pages: " << book1.pages << endl;

    cout << "\n--- Book 2 ---" << endl;
    cout << "Title: " << book2.title << endl;
    cout << "Author: " << book2.author << endl;
    cout << "Price: $" << book2.price << endl;
    cout << "Pages: " << book2.pages << endl;

    return 0;
}
```

5. CONCLUSION

This lab covered the implementation of user-defined data types using structures in C++. Structures provide a crucial mechanism for grouping logically related variables of differing data types under a single unified name. By completing tasks that involved creating structures for people, students, and books, the utility of organizing complex data into manageable, object-like entities was clearly demonstrated. This concept serves as a vital stepping stone toward understanding classes and Object-Oriented Programming (OOP) in C++.